

Robótica

Informe capítulo 2 – RVC v2 Peter Corke

Localización y Mapeo

Docente:

Jaime Enrique Arango Castro

Estudiante:

Cristhian Mateo Almeida Gómez

Universidad Nacional de Colombia

Sede Manizales



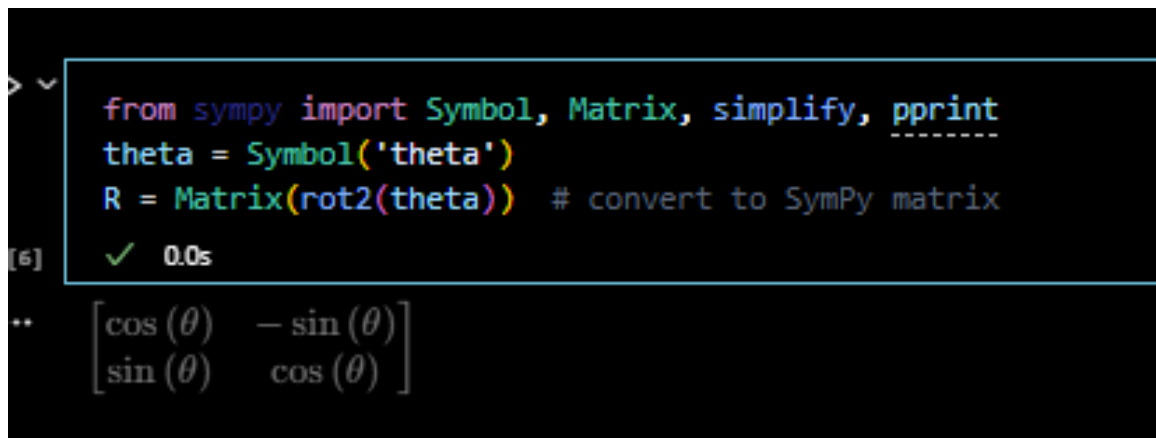
Resumen

El capítulo 2 del libro *Robotics, Vision and Control* se centra en el estudio de las matrices de rotación y su papel fundamental en la representación de orientaciones y movimientos en el espacio. Comienza con rotaciones en 2D y se extiende a rotaciones en 3D, utilizando diferentes convenciones como Euler, RPY y ángulos de orientación. También se introducen las matrices de transformación homogénea, que combinan rotación y traslación en una sola representación.

- **2.2.1.1 2D Matriz de rotación.**

En esta sección se explica cómo se representa una rotación en el plano mediante una matriz de rotación 2×2 . Esta matriz se utiliza para rotar puntos o vectores en el plano XY en ángulo, en sentido antihorario respecto al origen. La fórmula matemática involucra funciones trigonométricas y permite aplicar rotaciones con multiplicación matricial. En Python, esto se implementa fácilmente con la función `rot2()` del módulo `spatialmath.base`.

La matriz de rotación en 2D permite girar un vector en el plano alrededor del origen sin cambiar su longitud. Esta transformación se expresa mediante una matriz que depende del ángulo θ .



```
> ✓ from sympy import Symbol, Matrix, simplify, pprint
theta = Symbol('theta')
R = Matrix(rot2(theta)) # convert to SymPy matrix
[6] ✓ 0.0s
.. 
$$\begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix}$$

```

Figura 1. Matriz de rotación 2D

Al multiplicar esta matriz por un vector, se obtiene un nuevo vector que ha sido rotado en sentido antihorario por el ángulo θ . También debemos saber que La matriz gira el vector alrededor del origen, sin escalarlo ni deformarlo, Es una transformación ortogonal (con columnas ortogonales y de norma 1), lo que significa que conserva distancias y ángulos.

- **2.2.1.2 Matriz exponencial para rotación.**

En esta sección se introduce el concepto de exponencial de matrices como una forma alternativa de representar rotaciones. En particular, se muestra cómo una matriz de rotación 2D puede expresarse como la exponencial de una matriz antisimétrica escalar multiplicada por un ángulo.

Hay una conexión entre la matriz de rotación y la exponencial de una matriz antisimétrica. La exponencial de una matriz antisimétrica siempre es una matriz de rotación.

```

L = linalg.logm(R) # using linalg package of SciPy
✓ 0.0s
array([[ 0., -0.3],
       [ 0.3,  0.]])

linalg.expm(L)
✓ 0.0s
array([[ 0.9553, -0.2955],
       [ 0.2955,  0.9553]])

```

Figura 2. Matriz de antisimétrica y matriz de rotación.

2.2.2 Pose en dos dimensiones

- **2.2.2.1 Matriz de transformación homogénea 2D**

se introduce la matriz de transformación homogénea en 2D, que combina en una sola estructura la rotación y la traslación de un sistema de coordenadas. Esta matriz permite representar de forma compacta cómo se mueve un punto o un objeto rígido en el plano. La transformación se expresa como una matriz 3×3 que incluye una submatriz de rotación 2×2 y un vector de traslación, facilitando operaciones como concatenar movimientos o invertir transformaciones geométricas.

2.2.2 Pose in Two Dimensions

2.2.2.1 2D Homogeneous Transformation Matrix

```

rot2(0.3)
✓ 0.0s
array([[ 0.9553, -0.2955],
       [ 0.2955,  0.9553]])

trot2(0.3)
✓ 0.0s
array([[ 0.9553, -0.2955,  0],
       [ 0.2955,  0.9553,  0],
       [ 0, 0, 1]])

```

Figura 3. Matriz de transformación homogénea.

Vemos la matriz de rotación en la esquina superior izquierda, y ceros y un uno en la fila inferior. Los dos primeros elementos de la columna derecha son cero, lo que indica una traslación cero. Esta matriz representa la pose relativa y puede combinarse con otras poses relativas o utilizarse para transformar vectores de coordenadas homogéneas.

La función `trans12()` crea una pose relativa 2D con una traslación finita pero rotación cero, mientras que `trot2()` crea una pose relativa con una rotación finita pero traslación cero.

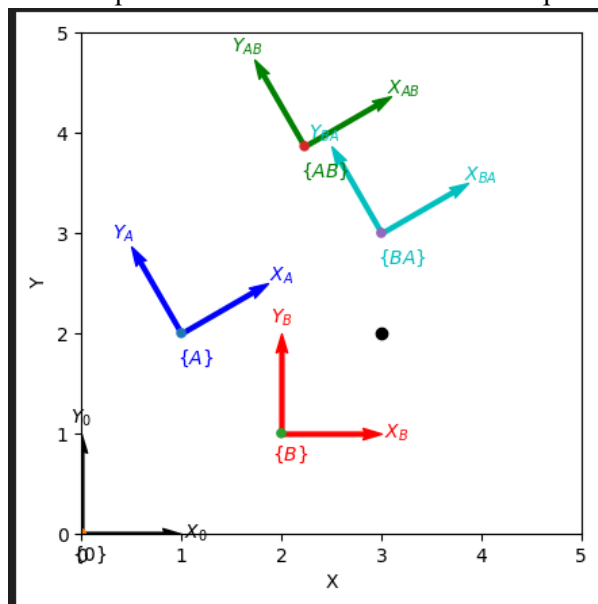


Figura 4. Marcos de coordenadas dibujados usando la función `trplot2`.

La figura 4 muestra distintos marcos de referencia coordenados en un plano 2D, ilustrando cómo se relacionan entre sí mediante transformaciones homogéneas. Los marcos $\{A\}$ y $\{B\}$ representan dos sistemas de coordenadas con sus respectivos ejes X_a, Y_a , también X_b, Y_b , junto con los vectores de transformación que permiten convertir posiciones y orientaciones de un marco a otro. Los marcos $\{AB\}$ y $\{BA\}$ representan las transformaciones relativas: $\{AB\}$ indica la posición de B respecto a A, mientras que $\{BA\}$ representa la posición de A respecto a B. La visualización también incluye el marco base $\{0\}$, mostrando que todos los sistemas están ubicados en el mismo espacio, pero con diferentes posiciones y orientaciones.

- 2.2.2.2 Rotación de un marco de coordenadas

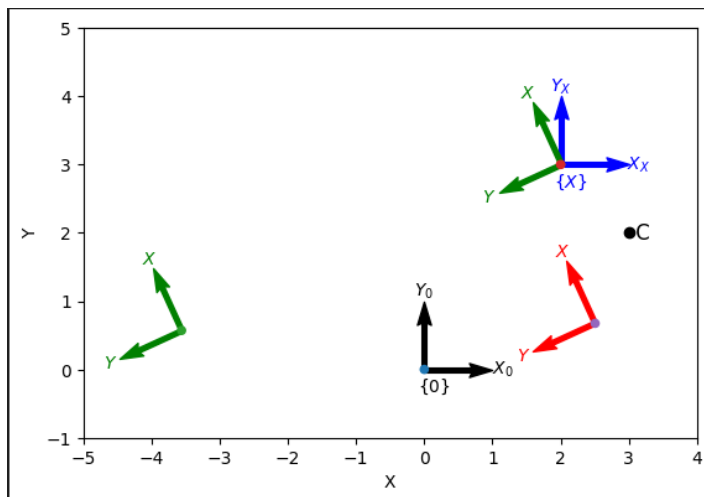


Figura 5. El marco $\{X\}$ se gira 2 radianes alrededor de $\{0\}$ para obtener el marco $\{RX\}$, alrededor de $\{X\}$ para obtener $\{XR\}$, y alrededor del punto C para obtener el marco $\{XC\}$.

Para comprender cómo funciona esto, podemos desglosar lo que sucede al aplicar TC a TX. Dado que TC pre multiplica TX, la primera transformación que se aplica a {X} es la de más a la derecha, $\text{transl2}(-C)$, que realiza un desplazamiento del origen que coloca a C en el origen del sistema de referencia. Luego, aplicamos TR, una rotación pura que rota la versión desplazada de {X} sobre el origen, que es donde se encuentra C ahora; por lo tanto, {X} se rota sobre C. Finalmente, aplicamos $\text{transl2}(C)$, que es el desplazamiento del origen inverso, que devuelve a C a su posición original y a {X} a su posición final. Esta transformación es un ejemplo de conjugación, pero su construcción fue algo elaborada.

- **2.2.2.3 Matriz exponencial para pose**

La transformación homogénea T en 2D también se puede expresar de la forma

$$T = e^{[S]}$$

Donde S es una matriz anti simétrica 3x3

- **2.2.2.4 Giros 2D**

Dados dos sistemas cualesquiera, podemos encontrar un centro de rotación y un ángulo de rotación que rotarán el primero hacia el segundo. Este es el concepto clave de lo que se denomina twist.

2.2.2.4 2D Twists

```

S = Twist2.UnitRevolute(C)
✓ 0.0s
(2 -3; 1)

linalg.expm(skewa(2 * S.S))
✓ 0.0s
array([[ -0.4161, -0.9093,  6.067],
       [  0.9093, -0.4161,  0.1044],
       [  0, 0, 1]])

S.exp(2)
✓ 0.0s
-0.4161 -0.9093  6.067
 0.9093 -0.4161  0.1044
 0 0 1

S.pole
✓ 0.0s
array([ 3, 2])

S = Twist2.UnitPrismatic((0, 1))
✓ 0.0s
(0 1; 0)

S.exp(2)
✓ 0.0s
1 0 0
0 1 2
0 0 1

```

```

T = transl2(3, 4) @ trot2(0.5)
✓ 0.0s
array([[ 0.8776, -0.4794,  3],
       [ 0.4794,  0.8776,  4],
       [ 0, 0, 1]])

S = Twist2(T)
✓ 0.0s
(3.9372 3.1663; 0.5)

S.w
✓ 0.0s
0.5

S.pole
✓ 0.0s
array([-3.166,  3.937])

S.exp(1)
✓ 0.0s
0.8776 -0.4794  3
0.4794  0.8776  4
0 0 1

```

Figura 6. Código twist2.

La función twist2 encapsula un vector de giro rotacional 2D (v,w) von v siendo una matriz de 2x2 y un escalar w. siendo s el centro de rotación, también llamado polo para una translación específica, la escalamos y la exponenciamos, de la misma forma podemos hacer una transformación homogénea arbitraria.

2.3 Trabajando en tres dimensiones (3D)

• 2.3.1 Orientación en tres dimensiones

introduce cómo representar la orientación de un cuerpo en el espacio 3D. Se exploran las matrices de rotación tridimensionales, que describen cómo girar un sistema de coordenadas respecto a los ejes X, Y y Z. Además, se presentan diferentes formas de describir estas rotaciones, como ángulos de Euler y RPY (roll, pitch, yaw), explicando su estructura, composición y cómo convertir entre ellas utilizando funciones en Python.

• 2.3.1.1 Matriz de rotación 3D

- se enfoca en las matrices de rotación en tres dimensiones, que son fundamentales para describir la orientación de objetos en el espacio tridimensional. Una matriz de rotación 3D es una matriz ortogonal de 3×3 con determinante igual a 1, que rota un vector alrededor de uno de los ejes coordenados (X, Y o Z) sin cambiar su magnitud. En esta sección se definen las funciones `rotx( $\theta$ )`, `roty( $\theta$ )` y `rotz( $\theta$ )` en Python, que generan las matrices de rotación respectivas alrededor de cada eje para un ángulo θ dado. Estas matrices pueden combinarse mediante multiplicación para describir rotaciones compuestas. Se utiliza código Python para visualizar estas rotaciones y para confirmar que las matrices cumplen propiedades como la ortogonalidad y la inversa siendo igual a la traspuesta, Se debe tener en cuenta que la composición o multiplicación de matrices no es conmutativa por lo que el orden resulta muy importante.

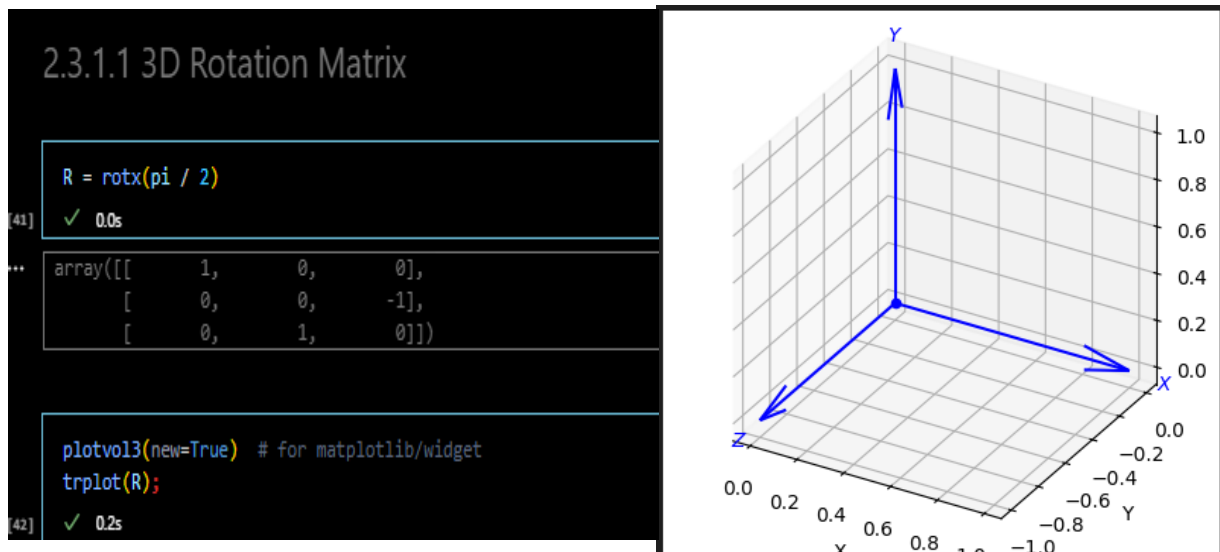
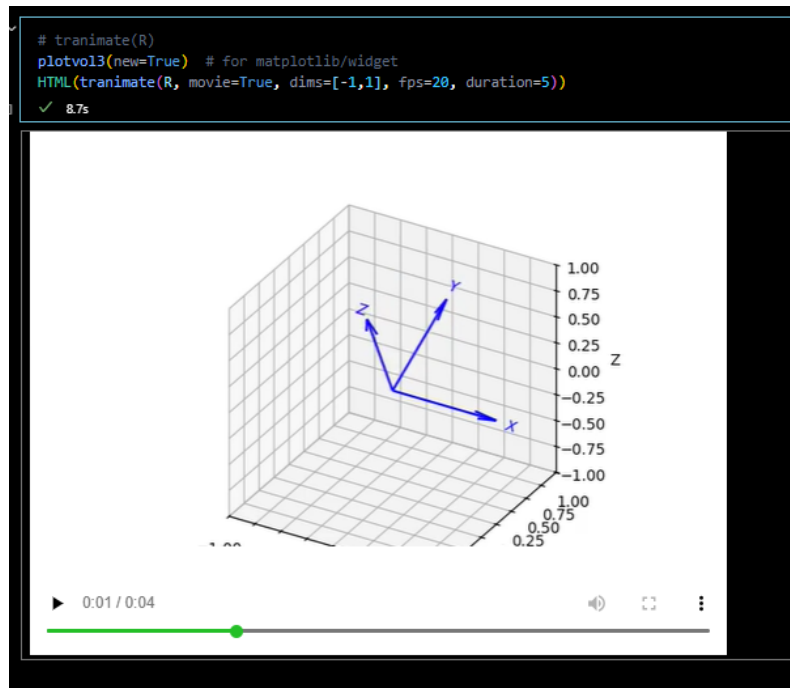


Figura 7. Marcos de coordenadas 3D mostrados con trplot.

Posteriormente se puede aplicar la transformación para visualizar por pantalla mediante una animación:



```
R = rotx(pi / 2) @ roty(pi / 2)
trplot(R);
✓ 0.0s
```

```
Ryx = roty(pi / 2) @ rotx(pi / 2)
✓ 0.0s
```

```
array([[ 0,  1,  0],
       [ 0,  0, -1],
       [-1,  0,  0]])
```

```
plotvol3(new=True) # for matplotlib/widget
trplot(Ryx);
✓ 0.1s
```

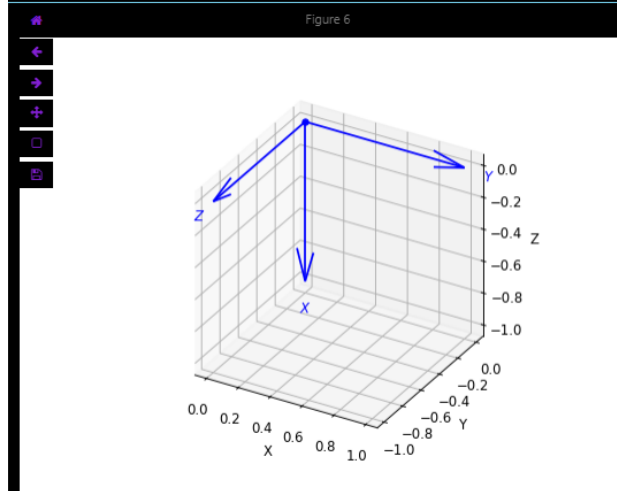


Figura 7.1 Transformaciones en el espacio.

- **2.3.1.2 Representaciones de tres ángulos**

Por una parte, El teorema de rotación de Euler significa que una rotación entre dos sistemas de coordenadas cualesquiera en 3D puede representarse mediante una secuencia de tres ángulos de rotación, cada uno asociado a un eje particular del sistema de coordenadas.

```
[49] R = rotz(0.1) @ roty(0.2) @ rotz(0.3);
      ✓ 0.0s

[50] R = eul2r(0.1, 0.2, 0.3)
      ✓ 0.0s

... array([[ 0.9021, -0.3836,  0.1977],
          [ 0.3875,  0.9216,  0.01983],
          [-0.1898,  0.05871,  0.9801]])
```

Figura 8. Matriz de rotación de Euler.

También se puede representar con los Ángulos de Tait-Bryan (o RPY), son rotaciones alrededor de tres ejes diferentes, típicamente Z-Y-X, también conocidos como yaw-pitch-roll.

```
R = rpy2r(0.1, 0.2, 0.3, order="zyx")
✓ 0.0s

array([[ 0.9363, -0.2751,  0.2184],
       [ 0.2896,  0.9564, -0.03696],
       [-0.1987,  0.09784,  0.9752]])
```

Figura 9. Matriz de rotación RPY.

- **2.3.1.3 Singularidades y bloqueo del cardán**

Una singularidad en este contexto es una configuración en la que el sistema pierde un grado de libertad. Esto significa que dos de los tres ejes de rotación se alinean, y por tanto no se puede distinguir entre ciertas rotaciones: la orientación se vuelve ambigua. Un Gimbal Lock ocurre cuando uno de los ángulos (típicamente el ángulo de pitch) alcanza $\pm 90^\circ$, y los ejes de dos de los giros se alinean. Al pasar por esta configuración, el sistema pierde efectivamente una dimensión de rotación.

- **2.3.1.4 Representación de dos vectores**

La orientación de un marco de coordenadas 3D se puede definir si conocemos, primero un vector de dirección principal (por ejemplo, el eje z del marco) y Un segundo vector que indique cómo está orientado otro eje (por ejemplo, una aproximación del eje x o y).

```
[62] a = [0, 0, -1]
      ✓ 0.0s
... [0, 0, -1]

[63] o = [1, 1, 0]
      ✓ 0.0s
... [1, 1, 0]

[64] R = oa2r(o, a)
      ✓ 0.0s
... array([[ -0.7071,  0.7071,  0],
          [ 0.7071,  0.7071,  0],
          [ 0, 0, -1]])
```

Figura 10. Representación de dos vectores.

Dos vectores no paralelos cualesquiera son suficientes para definir un sistema de coordenadas. Incluso si los dos vectores no son ortogonales, definen un plano, y el calculado es normal a ese plano.

• 2.3.1.5 Rotación sobre un vector arbitrario

la matriz de rotación 3D que rota un vector o un marco de referencia en el espacio alrededor de un eje arbitrario (definido por un vector) una cierta cantidad de ángulo θ .

El problema inverso, que convierte un ángulo y un vector en una matriz de rotación, se resuelve utilizando la fórmula de rotación de Rodrigues.

$$R = I_{3 \times 3} + \sin\theta[\hat{v}]_x + (1 - \cos\theta)[\hat{v}]_x^2$$

$\hat{v}$ es la matriz antisimétrica construida a partir del vector

```
R = angvec2r(0.3, [1, 0, 0])
✓ 0.0s
array([[ 1,  0,  0],
       [ 0, 0.9553, -0.2955],
       [ 0, 0.2955, 0.9553]])
```

Figura 11. Rotación sobre un vector arbitrario.

- **2.3.1.6 Matriz exponencial para rotación**

Primero, se crea una matriz de rotación sobre el eje x usando la función `rotx()` la cual es una rotación típica con matrices de rotación clásica esto se lo hace para compararlo mas adelante

```
R = rotx(0.3)
✓ 0.0s
array([[ 1,  0,  0],
       [ 0, 0.9553, -0.2955],
       [ 0, 0.2955, 0.9553]])

L = linalg.logm(R)
✓ 0.0s
array([[ 0,  0,  0],
       [ 0,  0, -0.3],
       [ 0, 0.3,  0]])

> S = vex(L)
✓ 0.0s
array([ 0.3,  0,  0])

L = trlog(R);
✓ 0.0s

linalg.expm(L)
✓ 0.0s
array([[ 1,  0,  0],
       [ 0, 0.9553, -0.2955],
       [ 0, 0.2955, 0.9553]])
```

Figura 12. Generación de matriz antisimétrica.

Se aplica la función `logm()` para obtener el logaritmo matricial de R, que nos devuelve una matriz antisimétrica, esta matriz contiene dos valores no nulos que indican un ángulo de rotación de 0.3 radianes alrededor del eje x.

También se puede escribir como:

```

linalg.expm(skew(S))
80] ✓ 0.0s
** array([[ 1,  0,  0],
          [ 0, 0.9553, -0.2955],
          [ 0, 0.2955, 0.9553]])

R = rotx(0.3);
81] ✓ 0.0s

R = linalg.expm(0.3 * skew([1, 0, 0]));
82] ✓ 0.0s

X = skew([1, 2, 3])
83] ✓ 0.0s
** array([[ 0, -3,  2],
          [ 3,  0, -1],
          [-2,  1,  0]])

vex(X)
84] ✓ 0.0s
** array([ 1,  2,  3])

```

Figura 13. representación de la matriz de rotación.

Aquí se representa una rotación de 0.3 radianes alrededor del eje unitario $\hat{w} = [1,0,0]$, tenemos que tener claro que una rotación en $SO(3)$ puede representarse como:

$$R = e^{\theta \hat{w}}$$

Donde $\hat{w}$ es la matriz antisimétrica (skew) del eje unitario de rotación y θ es el ángulo de rotación.

- **2.3.1.7 Cuaterniones unitarios**

Un cuaternión es una extensión de los números complejos con cuatro componentes

```
q = UnitQuaternion(rpy2r(0.1, 0.2, 0.3))
✓ 0.0s
0.9833 << 0.0343, 0.1060, 0.1436 >>

q = q * q;
✓ 0.0s

q.inv()
✓ 0.0s
0.9339 << -0.0674, -0.2085, -0.2824 >>

q * q.inv()
✓ 0.0s
1.0000 << 0.0000, 0.0000, 0.0000 >>

q / q
✓ 0.0s
1.0000 << 0.0000, 0.0000, 0.0000 >>

q.R
✓ 0.0s
array([[ 0.7536, -0.4993, 0.4275],
       [ 0.5555, 0.8315, -0.008145],
       [-0.3514, 0.2436, 0.904]])

q * [1, 0, 0]
✓ 0.0s
array([ 0.7536, 0.5555, -0.3514])
```

Figura 14. cuaternios y sus propiedades.

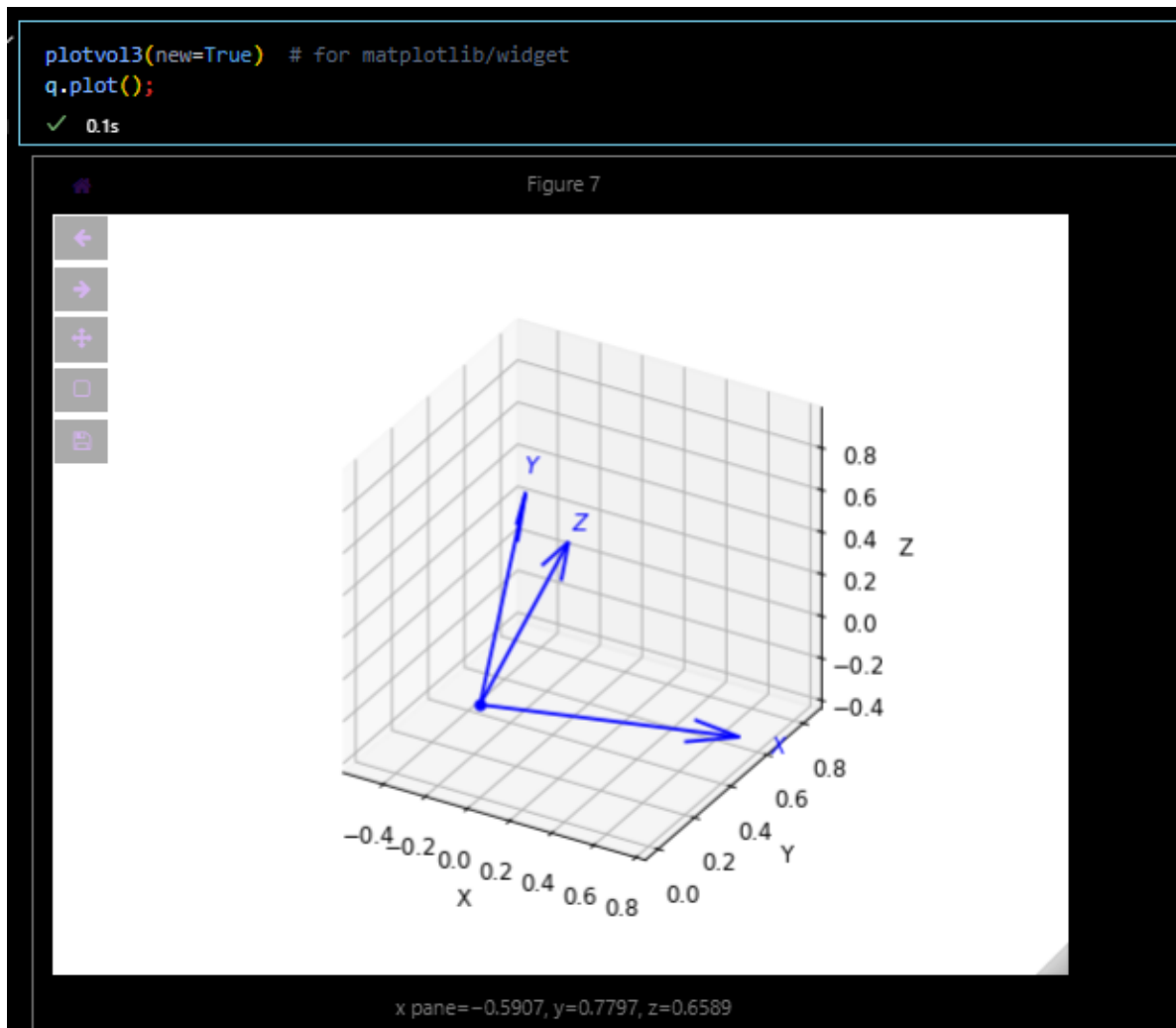


Figura 14.1. Transformación mediante cuaterniones.

Esto crea un cuaternión unitario a partir de una rotación definida por ángulos roll, pitch y yaw. También tiene propiedades como multiplicación, inverso, rotar en un vector, convertir a matriz de rotación y graficación. De igual manera, se pueden aplicar transformaciones mediante estos con el fin de evitar singularidades.

2.3.2 Pose en Tres Dimensiones

Cuando trabajamos en 3D, describir la pose relativa entre dos sistemas de referencia implica tener en cuenta tanto Rotación (la orientación de un marco respecto a otro) y Traslación (el desplazamiento entre los orígenes de los marcos).

- **2.3.2.1 Matriz de transformación homogénea**

la matriz 4x4 de transformación homogénea es una transformación rígida de rotación y traslación

$$\begin{pmatrix} A_x \\ A_y \\ A_z \\ 1 \end{pmatrix} = \begin{pmatrix} {}^A\mathbf{R}_B & {}^A\mathbf{t}_B \\ \mathbf{0}_{1 \times 3} & 1 \end{pmatrix} \begin{pmatrix} B_x \\ B_y \\ B_z \\ 1 \end{pmatrix}$$

Figura 15. Matriz de transformación homogénea.

The screenshot shows a Jupyter Notebook interface with the following content:

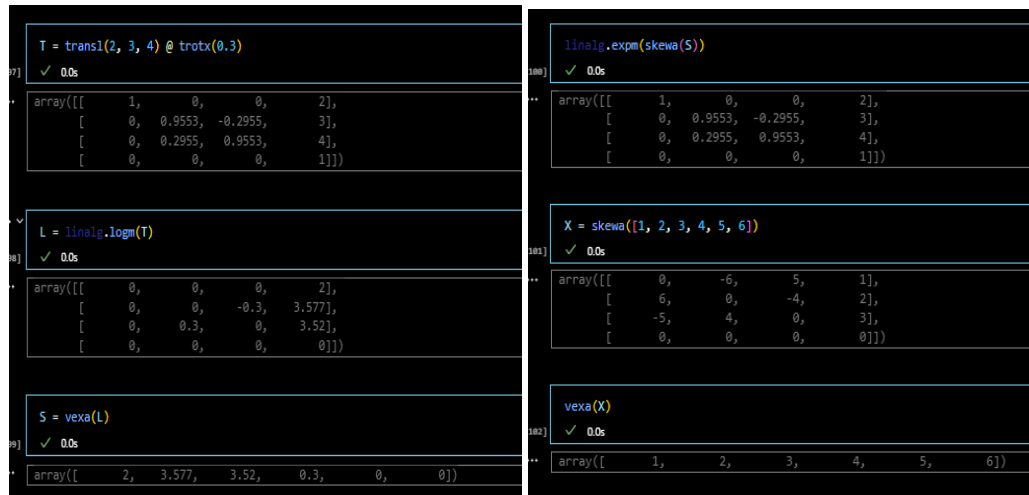
- Section header: 2.3.2 Pose in Three Dimensions
- Section header: 2.3.2.1 Homogeneous Transformation Matrix
- Code cell input: `T = transl(2, 0, 0) @ trotx(pi / 2) @ transl(0, 1, 0)`
- Code cell output: `array([[ 1, 0, 0, 2], [ 0, 0, -1, 0], [ 0, 1, 0, 1], [ 0, 0, 0, 1]])`

Figura 16. Matriz de transformación homogénea.

Esto representa trasladarse 2 unidades sobre el eje x, rotar 90° sobre el eje x, por ultimo trasladarse 1 unidad sobre el nuevo eje y.

Como funciones claves tenemos `t2r(T)` extrae la parte de rotación (3×3) y `transl(T)` extrae la parte de traslación (3×1).

- **2.3.2.2 Matriz exponencial para pose**



```

T = transl(2, 3, 4) @ trotx(0.3)
array([[ 1, 0, 0, 2],
       [ 0, 0.9553, -0.2955, 3],
       [ 0, 0.2955, 0.9553, 4],
       [ 0, 0, 0, 1]])

L = linalg.logm(T)
array([[ 0, 0, 0, 2],
       [ 0, 0, -0.3, 3.577],
       [ 0, 0.3, 0, 3.52],
       [ 0, 0, 0, 0]])

S = vexa(L)
array([ 2, 3.577, 3.52, 0.3, 0, 0])

linalg.expm(skewa(S))
array([[ 1, 0, 0, 2],
       [ 0, 0.9553, -0.2955, 3],
       [ 0, 0.2955, 0.9553, 4],
       [ 0, 0, 0, 1]])

X = skewa([1, 2, 3, 4, 5, 6])
array([[ 0, -6, 5, 1],
       [ 6, 0, -4, 2],
       [-5, 4, 0, 3],
       [ 0, 0, 0, 0]])

vexa(X)
array([ 1, 2, 3, 4, 5, 6])

```

Figura 17. Matriz exponencial para pose.

Primero se crea una matriz de transformación homogénea T , después con el comando $L = \text{linalg.logm}(T)$ calcula el logaritmo de matriz, que da una representación del movimiento como una matriz aumentada antisimétrica, cuando hacemos a $S = \text{vexa}(L)$ nos devuelve un vector donde los tres primeros componentes son los de traslación y los otros tres los de rotación. Por último, se nos permite hacer una reconstrucción con la exponencial.

En general, cualquier transformación homogénea en 3D puede escribirse como:

$$T = e^{\hat{s}}$$

Donde $\hat{s}$ es una matriz 4x4 aumentada skew-symmetric.

• 2.3.2.3 Giros 3D

Un twist en 3D describe un movimiento helicoidal (como el de un tornillo), una combinación de rotación alrededor de un eje y traslación a lo largo del mismo eje.

```

183] S = Twist3.UnitRevolute([1, 0, 0], [0, 0, 0])
✓ 0.0s
(0 0 0; 1 0 0)

184] linalg.expm(0.3 * skewa(S.S)); # different to book, see §2.2.1.
✓ 0.0s

185] S.exp(0.3)
✓ 0.0s
1      0      0      0
0      0.9553 -0.2955  0
0      0.2955  0.9553  0
0      0      0      1

186] S = Twist3.UnitRevolute([0, 0, 1], [2, 3, 2], 0.5)
✓ 0.0s
(3 -2 0.5; 0 0 1)

187] X = transl(3, 4, -4);
✓ 0.0s

188] for theta in np.arange(0, 15, 0.3):
    trplot(S.exp(theta).A @ X, style="rviz", width=2)

    L = S.line()
    L.plot('k:', linewidth=2);
✓ 0.3s
Axes3D(0.22375,0.11;0.5775x0.77)

```

Figura 18. Twists.

Esto crea un twist unitario con un eje de rotación paralelo a x y el punto por donde pasa el eje el origen. Para generar el movimiento Helicoidal con coordenadas $X = \text{transl}(3, 4, -4)$ rotando sobre un eje paralelo al eje z , pasando por el punto $(2, 3, 2)$ con una pitch de 0.5 con un ciclo de rotación progresiva.


```

S = Twist3.UnitPrismatic([0, 1, 0])
[109] ✓ 0.0s
... (0 1 0; 0 0 0)

S.exp(2)
[110] ✓ 0.0s
...
1      0      0      0
0      1      0      2
0      0      1      0
0      0      0      1

T = transl(1, 2, 3) @ eul2tr(0.3, 0.4, 0.5);
S = Twist3(T)
[111] ✓ 0.0s
... (1.1204 1.6446 3.1778; 0.041006 0.4087 0.78907)

S.w
[112] ✓ 0.0s
... array([ 0.04101,  0.4087,  0.7891])

```

Figura 19. Ejemplo twist prismático (traslación).

Podemos hacer un Twist de traslación en la dirección del eje Y , si aplicamos $S.exp(2)$ el resultado será una matriz sin rotación.

```

S.pole
[113] ✓ 0.0s
... array([0.001138,  0.8473, -0.4389])

S.pitch
[114] ✓ 0.0s
... 3.2256216289351296

S.theta
[115] ✓ 0.0s
... 0.8895797456112914

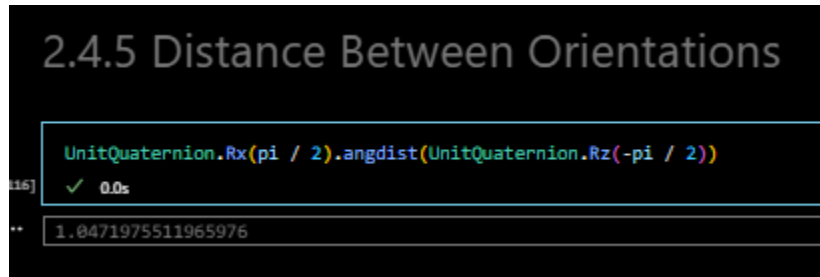
```

Figura 20. Convertir SE(3) a Twist

Esto indica que el movimiento puede representarse como un eje helicoidal con dirección $S.w$, un punto en el eje $S.pole$, un paso helicoidal ($S.pitch = 3.226$) o un ángulo de giro ($S.theta = 0.8896$ rad $\approx 51^\circ$).

2.4 Temas avanzados

- 2.4.5 Distancia entre Orientaciones



```
UnitQuaternion.Rx(pi / 2).angdist(UnitQuaternion.Rz(-pi / 2))
```

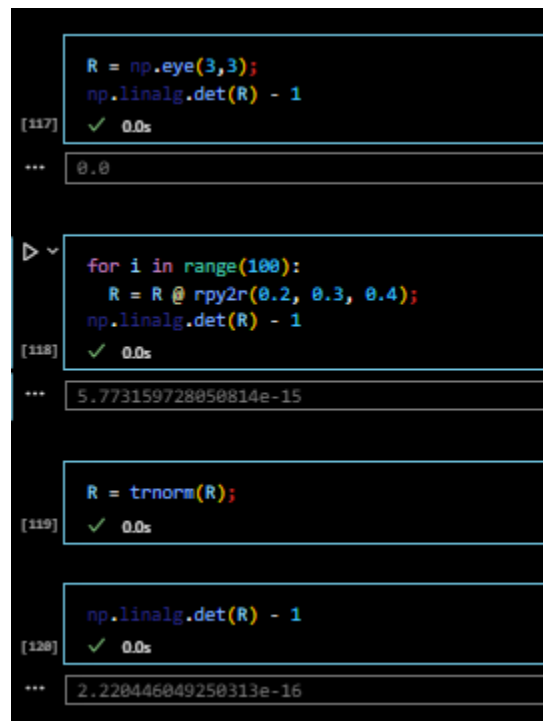
[116] ✓ 0.0s

... 1.0471975511965976

Figura 21. Distancia entre orientaciones.

Rotación de 90° sobre x contra 90° sobre z, con una Distancia angular $\approx 60^\circ$ o 1.047 rad.

- 2.4.6 Normalización



```
R = np.eye(3,3);  
np.linalg.det(R) - 1
```

[117] ✓ 0.0s

... 0.0

```
for i in range(100):  
    R = R @ rpy2r(0.2, 0.3, 0.4);  
    np.linalg.det(R) - 1
```

[118] ✓ 0.0s

... 5.773159728050814e-15

```
R = trnorm(R);
```

[119] ✓ 0.0s

```
np.linalg.det(R) - 1
```

[120] ✓ 0.0s

... 2.220446049250313e-16

Figura 22. normaliza la matriz de rotación.

Con el ciclo se debería producir una matriz de rotación válida, pero una matriz de rotación válida debe cumplir Ser ortogonal y su determinante debe ser 1. Este problema se lo soluciona con la normalización de la matriz. Con `tnorm()` normaliza la matriz de rotación

• 2.4.8 Más sobre los giros

El teorema de Chasles dice que Cualquier desplazamiento de un cuerpo rígido en el espacio puede lograrse mediante una rotación alrededor de una línea (un eje) y una traslación a lo largo de esa línea.

Figura 23. Twists 3D

```

S = Twist3.UnitRevolute([1, 0, 0], [0, 0, 0])
✓ 0.0s
(0 0 0; 1 0 0)

trexp(0.3 * S.skewa())
✓ 0.0s
array([[ 1, 0, 0, 0],
       [ 0, 0.9553, -0.2955, 0],
       [ 0, 0.2955, 0.9553, 0],
       [ 0, 0, 0, 1]])

S.exp(0.3)
✓ 0.0s
1      0      0      0
0      0.9553 -0.2955  0
0      0.2955  0.9553  0
0      0      0      1

S2 = S * S
S2.printline(orient="angvec", unit="rad")
✓ 0.0s
t = 0, 0, 0; angvec = (2 | 1, 0, 0)

line = S.line()
✓ 0.0s
{ 0 0 0; 1 0 0}

plotvol3([-5, 5], new=True) # setup volume in which to display the line
line.plot("k:", linewidth=2);
✓ 0.4s
Axes3D(0.125,0.11;0.775x0.77)

```

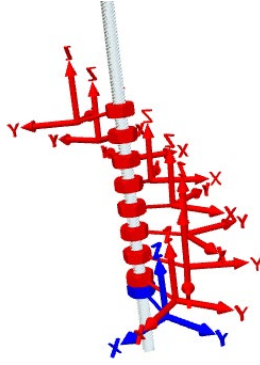


Figura 24. Aplicación de twists en 3D.

El teorema de Chasles dice que Cualquier desplazamiento de un cuerpo rígido en el espacio puede lograrse mediante una rotación alrededor de una línea (un eje) y una traslación a lo largo de esa línea.

El twist tiene dos componentes: una lineal (v), que representa la traslación, y una angular (w), que representa la rotación. En este caso, v es cero (no hay traslación) y w es $1, 0, 0, 0, 0, 1, 0, 0, 0$.

Podemos ver la forma matricial del twist utilizando $SE(3)$. Esta función devuelve una matriz 4×4 que pertenece al álgebra de Lie $SE(3)$. Dicha matriz puede ser exponenciada para obtener una transformación $T \in SE(3)$.

Si multiplicamos la matriz $se3()$ por un escalar (por ejemplo, 0.3) y aplicamos $trexp()$, obtenemos una matriz de transformación homogénea. Esto es equivalente a usar funciones como $trotx(0.3)$.

Una forma más directa es usar $S.exp(0.3)$, que también devuelve la transformación correspondiente a una rotación de 0.3 radianes alrededor del eje x .

También podemos escalar un twist multiplicándolo por un escalar: $S2 = S * S$. Esto da como resultado un nuevo twist que representa una rotación de 2 radianes alrededor del mismo eje.

```

T = transl(1, 2, 3) @ eul2tr(0.3, 0.4, 0.5);
S = Twist3(T)
[131] ✓ 0.0s
... (1.1204 1.6446 3.1778; 0.041006 0.4087 0.78907)

S / S.theta
[132] ✓ 0.0s
... (1.2594 1.8488 3.5722; 0.046096 0.45943 0.88702)

S.unit();
[133] ✓ 0.0s

S.exp(0)
[134] ✓ 0.0s
...
1      0      0      0
0      1      0      0
0      0      1      0
0      0      0      1

S.exp(1)
[135] ✓ 0.0s
...
0.6305  -0.6812   0.372   1
0.6969   0.7079   0.1151   2
-0.3417   0.1867   0.9211   3
0         0         0         1

S.exp(0.5)
[136] ✓ 0.0s
...
0.9029  -0.3796   0.2017   0.5447
0.3837   0.9232   0.01982  0.9153
-0.1937   0.05949  0.9793   1.542
0         0         0         1

```

Figura 25. Transformación homogénea y exponencial.

Aquí, S es el twist cuya exponencial nos da la transformación T. La magnitud del twist se guarda en S.theta. Podemos obtener el twist unitario dividiendo por S.theta o usando S.unit(). Si aplicamos S.exp(0), obtenemos la matriz identidad (sin movimiento). Si aplicamos S.exp(1), recuperamos la transformación original. Y con S.exp(0.5), obtenemos la mitad del movimiento.

• 2.5 Uso de la caja de herramientas

La Spatial Maths Toolbox tiene dos niveles. El primero, o nivel base, se accede con spatialmath.base esto importa funciones que trabajan directamente con matrices NumPy, como

transl2, rotx, y trexp, además de funciones para cuaterniones, gráficos (trplot, plot_point) y utilidades generales.

El segundo nivel es la librería spatialmath. Este nivel proporciona clases más avanzadas, como SO3, SE3, UnitQuaternion, Twist2, Twist3, Line3, etc. Estas clases encapsulan matrices y ofrecen métodos especializados para rotaciones y transformaciones.

```
[138] from spatialmath.base import *
✓ 0.0s

[139] from spatialmath import *
✓ 0.0s

R = rotx(0.3) # create SO(3) matrix as NumPy array
type(R)
R = SO3.Rx(0.3) # create SO3 object
type(R)
[140] ✓ 0.0s
... spatialmath.pose3d.SO3

R.A
[141] ✓ 0.0s
... array([[ 1,      0,      0],
          [ 0,  0.9553, -0.2955],
          [ 0,  0.2955,  0.9553]])
```

```

R = SO3(rotx(0.3));           # convert an SO(3) matrix
R = SO3.Rz(0.3);             # rotation about z-axis
R = SO3.RPY(10, 20, 30, unit="deg"); # from roll-pitch-yaw angles
R = SO3.AngleAxis(0.3, (1, 0, 0)); # from angle and rotation axis
R = SO3.EulerVec((0.3, 0, 0)); # from an Euler vector
42] ✓ 0.0s

R.rpy(); # convert to roll-pitch-yaw angles
R.eul(); # convert to Euler angles
R.println(); # compact single-line print
43] ✓ 0.0s

rpy/zyx = 17.2°, 0°, 0°

R = SO3.RPY(10, 20, 30, unit="deg"); # create an SO(3) rotation
T = SE3.RPY(10, 20, 30, unit="deg"); # create a purely rotational SE(3)
S = Twist3.RPY(10, 20, 30, unit="deg"); # create a purely rotational twist
q = UnitQuaternion.RPY(10, 20, 30, unit="deg"); # create a unit quaternion
44] ✓ 0.0s

TA = SE2(1, 2) * SE2(30, unit="deg");
type(TA)
45] ✓ 0.0s

spatialmath.pose2d.SE2

TA
46] ✓ 0.0s

0.866    -0.5    1
0.5      0.866    2
0         0        1

TA = SE2(1, 2, 30, unit="deg");
47] ✓ 0.0s

TA.R
TA.t
48] ✓ 0.0s

array([ 1, 2])

```

Figura 26. Uso de la caja de Herramientas.

Por ejemplo con `rotx()` obtenemos una matriz NumPy de 3×3 . En cambio, con `SO3.Rx` obtenemos un objeto `SO3`, que representa explícitamente una rotación en el espacio tridimensional. Aunque la representación visual es diferente, ambos almacenan la misma información. Podemos acceder a la matriz interna con `R.A`.

```

R = SO3(rotx(0.3));           # convert an SO(3) matrix
R = SO3.Rz(0.3);             # rotation about z-axis
R = SO3.RPY(10, 20, 30, unit="deg"); # from roll-pitch-yaw angles
R = SO3.AngleAxis(0.3, (1, 0, 0)); # from angle and rotation axis
R = SO3.EulerVec((0.3, 0, 0)); # from an Euler vector
142] ✓ 0.0s

R.rpy(); # convert to roll-pitch-yaw angles
R.eul(); # convert to Euler angles
R.println(); # compact single-line print
143] ✓ 0.0s

*** rpy/zyx = 17.2°, 0°, 0°

R = SO3.RPY(10, 20, 30, unit="deg"); # create an SO(3) rotation
T = SE3.RPY(10, 20, 30, unit="deg"); # create a purely rotational SE(3)
S = Twist3.RPY(10, 20, 30, unit="deg"); # create a purely rotational twist
q = UnitQuaternion.RPY(10, 20, 30, unit="deg"); # create a unit quaternion
144] ✓ 0.0s

```

Figura 27. Herramientas.

```

R = SO3.Rx(np.linspace(0, 1, 5));
len(R)
R[3]
152] ✓ 0.0s

***
1      0      0
0      0.7317 -0.6816
0      0.6816 0.7317

R * [1, 2, 3]
153] ✓ 0.0s

***
array([[ 1,      1,      1,      1,      1],
       [ 2,  1.196, 0.3169, -0.5815, -1.444],
       [ 3,  3.402, 3.592,  3.558,  3.304]])

```

Figura 28. Ejemplo de datos tipo objeto.

Esto crea 5 matrices de rotación con ángulos entre 0 y 1 rad, y si aplicamos estas rotaciones a un vector el resultado es una matriz donde cada columna es el vector rotado por cada rotación de R.

• Conclusiones

- La transformación homogénea permite describir posiciones y orientaciones.

- Twists y cuaterniones complementan la modelación en 3D.
- La Toolbox Python facilita experimentar con la POSE de un robot en el espacio.

- **Referencias**

- Chap2AdvanceNotes.pdf
- Corke, P. Robotics, Vision and Control v2 (RVC)
- notebook : chap2.ipynb
- Documentación Robotics Toolbox
- Git hub RVC v2